YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT A CONSTITUENT UNIT OF YENEPOYA (DEEMED TO BE UNIVERSITY) BALMATTA, MANGALORE

HIGH LEVEL DESIGN

ON

CLOUD COST INTELLIGENCE FOR E-COMMERCE

SUBMITTED BY

MUHAMMAD NIHAL

REG NO :

23BCAICD076

LH-06

GUIDED BY

MISS SINDHU

SUBMISSION DATE

17-04-2026

Table of Contents

1 Introduction
- 1.1 Scope of the document
- 1.2 Intended Audience
- 1.3 System overview

2 System Design
- 2.1 Application Design
- 2.2 Process Flow
- 2.3 Information Flow
- 2.4 Components Design
- 2.5 Key Design Considerations
- 2.6 API Catalogue

3 Data Design
- 3.1 Data Model
- 3.2 Data Access Mechanism
- 3.3 Data Retention Policies
- 3.4 Data Migration

4 Interfaces

5 State and Session Management

6 Caching

7 Non-Functional Requirements
- 7.1 Security Aspects
- 7.2 Performance Aspects

8 References

1. Introduction

1.1 Scope of the Document

This document describes the high-level design of the **Cloud Cost Intelligence System for eCommerce**. It outlines how the system is structured, how data flows through different components, and how insights such as anomalies and cost optimization recommendations are generated.

The focus is on giving a clear understanding of the system architecture without going into low-level coding details.

1.2 Intended Audience

This document is intended for developers, project reviewers, and stakeholders who need a clear understanding of how the system is designed. It is also useful for academic evaluation, where both technical clarity and structured documentation are required.

1.3 System Overview

The Cloud Cost Intelligence System is designed to monitor and analyze cloud usage costs in an eCommerce environment.

It processes raw usage data and transforms it into meaningful insights such as:

- Total cloud cost distribution
- Identification of wasteful resource usage
- Detection of unusual cost spikes (anomalies)
- Actionable recommendations for optimization

The system presents all insights through an interactive dashboard, making it easier for users to understand and act on cost-related issues.

2. System Design

2.1 Application Design

The system follows a simple yet effective architecture:

- **Frontend:** Built using Streamlit for interactive visualization
- **Backend:** Python-based data processing using Pandas
- **Machine Learning Layer:** Isolation Forest algorithm for anomaly detection
- **Data Source:** CSV dataset

This design ensures ease of development, fast execution, and good visualization capabilities.

2.2 Process Flow

The system follows a step-by-step pipeline:

1. The dataset is loaded into the system
2. Cost-related features are calculated from raw usage data
3. User-selected filters are applied
4. Waste detection logic identifies inefficient resource usage
5. Machine learning model detects anomalies
6. Recommendations are generated based on rules and model output
7. Results are displayed in the dashboard

This structured flow ensures clarity and modular processing.

2.3 Information Flow

Data flows in a linear and controlled manner:

- Input data is read from a CSV file
- It is processed and enriched with additional features
- Analytical models generate insights
- Final output is presented visually

This ensures that raw data is progressively transformed into actionable information.

2.4 Components Design

The system is divided into the following major components:

- **Data Loading Module:** Reads and initializes dataset
- **Feature Engineering Module:** Computes cost-related metrics
- **Waste Detection Module:** Identifies inefficient usage patterns
- **Anomaly Detection Module:** Detects unusual cost behavior
- **Recommendation Engine:** Suggests optimization actions
- **Visualization Layer:** Displays all insights in dashboard format

Each component has a specific responsibility, making the system modular and easy to maintain.

2.5 Key Design Considerations

While designing the system, the following factors were considered:

- Simplicity and readability of implementation

- Ability to handle moderate-sized datasets efficiently
- Accuracy of anomaly detection
- User-friendly dashboard design
- Flexibility for future enhancements

2.6 API Catalogue

The system does not use external APIs. All computations and logic are implemented internally using Python functions and libraries.

3. Data Design

3.1 Data Model

The dataset contains cloud usage parameters such as compute hours, storage usage, API calls, and active users.

From these, additional fields are derived:

- Individual cost components
- Total cost
- Waste indicators
- Anomaly labels
- Recommendations

This structured data model enables efficient analysis and decision-making.

3.2 Data Access Mechanism

Data is loaded using Pandas and processed entirely in memory. This approach is suitable for the current scale and ensures fast execution.

3.3 Data Retention Policies

The system does not store data permanently. All data is processed temporarily during execution and cleared after the session ends.

3.4 Data Migration

No data migration is required, as the system operates on static input files.

4. Interfaces

The system provides an interactive dashboard interface using Streamlit. Users can:

- Filter data by department
- View cost breakdowns
- Analyze anomalies and waste
- Explore recommendations

5. State and Session Management

Session handling is managed by Streamlit. User selections (such as filters) persist during the session but are not stored permanently.

6. Caching

Caching is not currently implemented. However, it can be added in the future to improve performance for larger datasets.

7. Non-Functional Requirements

7.1 Security Aspects

The system does not handle sensitive data and runs locally, reducing security risks.

7.2 Performance Aspects

The system is optimized for quick data processing using efficient Pandas operations and lightweight machine learning models.

8. References

- Streamlit Documentation
- Scikit-learn Documentation
- Pandas Documentation